CHANDIGARH UNIVERSITY

DISCOVER. LEARN. EMPOWER.

OFFERLY

CAMPUS PLACEMENT APPLICATION TRACKER

A PROJECT REPORT

Submitted by

YASH NALANDWAR (24BET10111)

VANSH (24BET10093)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Under the supervision of

Navdeep Kaur

Chandigarh University

Academic Session 2026–2027

https://github.com/Yash1911ind/offerly

BONAFIDE CERTIFICATE

Certified that this project report entitled "OFFERLY: CAMPUS PLACEMENT APPLICATION TRACKER" is the bonafide work of YASH NALANDWAR (UID: 24BET10111) and VANSH (UID: 24BET10093), who carried out the project work under the guidance and supervision of the undersigned.

This project report is submitted in partial fulfilment of the requirements of the course Full Stack Development – II (24BET_TPP-603) for the degree of Bachelor of Engineering in Information Technology at Chandigarh University.

STUDENT SIGNATURES	SUPERVISOR SIGNATURE
_____ YASH NALANDWAR UID: 24BET10111 _____ VANSH UID: 24BET10093	_____ Name: Navdeep Kaur Academic Designation: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to Chandigarh University and the Department of Information Technology for providing the academic environment and laboratory facilities required to undertake this project.

We are deeply thankful to our project supervisor, Navdeep Kaur, for the guidance, feedback and encouragement provided throughout the planning and development of Offerly. The concepts taught in the nine laboratory experiments of Full Stack Development – II formed the foundation on which this project was built, and the suggestions received during the course helped us understand the importance of requirement analysis, modular design, secure authentication, database integrity and systematic validation.

We also acknowledge the support of faculty members and classmates who reviewed the application and shared suggestions during its development. Finally, we thank our families for their constant encouragement and support.

YASH NALANDWAR (24BET10111)

VANSH (24BET10093)

TABLE OF CONTENTS

BONAFIDE CERTIFICATE 2

ACKNOWLEDGEMENT 3

LIST OF FIGURES 6

LIST OF TABLES 7

ABSTRACT 8

GRAPHICAL ABSTRACT 9

ABBREVIATIONS 10

SYMBOLS AND NOTATION 11

CHAPTER 1 INTRODUCTION 12

1.1 Client and Need Identification 12

1.2 Contemporary Context and Problem Identification 13

1.3 Identification of Tasks 13

1.4 Project Objectives and Scope 14

1.5 Timeline 15

1.6 Organization of the Report 15

CHAPTER 2 DESIGN FLOW / PROCESS 17

2.1 Concept Generation 17

2.2 Evaluation and Selection of Specifications / Features 17

2.3 Design Constraints 18

2.4 Analysis and Feature Finalization Subject to Constraints 19

2.5 Alternative Design Approaches 19

2.6 Design Selection 20

2.7 System Architecture and Data Flow 21

2.8 Database Design 23

2.9 Implementation Methodology 24

CHAPTER 3 RESULTS ANALYSIS AND VALIDATION 26

3.1 Implementation Environment and Modern Engineering Tools 26
3.2 Frontend Implementation 26
3.3 Backend Implementation 27
3.4 Authentication and Authorization 28
3.5 Application Integrity and Business Rules 29
3.6 API and Database Validation 30
3.7 Results Interpretation 32
3.8 Mapping to the Nine Lab Experiments 33
CHAPTER 4 CONCLUSION AND FUTURE WORK 34
4.1 Conclusion 34
4.2 Limitations and Deviations from Expected Results 34
4.3 Future Work 35
4.4 Way Ahead 35
REFERENCES 36
APPENDIX 1 – USER MANUAL 37
A1.1 Prerequisites 37
A1.2 Installation 37
A1.3 Running the application 37
A1.4 Student workflow 38
A1.5 Placement-cell workflow 38
A1.6 Troubleshooting 39
A1.7 Safe operation 39
APPENDIX 2 – SCREENSHOT EVIDENCE 40
APPENDIX 3 – SOURCE STRUCTURE AND ACHIEVEMENTS 46
A3.1 Source code structure 46
A3.2 Achievements 46

LIST OF FIGURES

Figure GA.1. Conceptual architecture and application lifecycle of Offerly 9

Figure 1.1. Sequence of project development activities 15

Figure 2.1. Three-tier architecture of Offerly 21

Figure 2.2. Process flow when a student applies for a job 22

Figure 2.3. Entity relationship overview 24

Figure 3.1. Output of the automated API test script (29 of 29 passed) 32

Figure A2.1. Home page with search and live placement statistics 40

Figure A2.2. Job listing with search and filters 40

Figure A2.3. Registration page for students and the placement cell 41

Figure A2.4. Login page 41

Figure A2.5. Job details with the application form 42

Figure A2.6. Job details after applying, showing the application status 42

Figure A2.7. My Applications page with status tracking 43

Figure A2.8. Application rejected by the CGPA eligibility rule 43

Figure A2.9. Placement-cell dashboard with statistics and job management 44

Figure A2.10. Form to publish a new job 44

Figure A2.11. All applications with status updates 45

Figure A2.12. Dashboard in dark mode 45

LIST OF TABLES

Table P.1. Abbreviations used in this report 10

Table P.2. Symbols and notation 11

Table 1.1. Problem dimensions and requirements 13

Table 1.2. Project tasks and deliverables 14

Table 2.1. Core use cases 17

Table 2.2. Candidate features and evaluation 18

Table 2.3. Design constraints and responses 19

Table 2.4. Alternative architecture comparison 20

Table 2.5. User collection fields 23

Table 2.6. Job collection fields 23

Table 2.7. Application collection fields 24

Table 2.8. Entity relationships 24

Table 3.1. Implementation environment 26

Table 3.2. REST API summary 28

Table 3.3. Business rules and enforcement 30

Table 3.4. Validation test cases and observed results 31

Table 3.5. Mapping of project work to the nine lab experiments 33

Table A1.1. Demo accounts created by the seed data 37

Table A1.2. Troubleshooting 39

ABSTRACT

Offerly is a full-stack web application that digitises the campus placement process of a university. In many colleges, job and internship opportunities are still announced through scattered notices, messaging groups and e-mails, while applications are collected through spreadsheets or forms. This causes missed opportunities, repeated applications from the same student, manual eligibility checking by the placement cell, and no single place where students can see the status of their applications. Offerly addresses these problems with one secure, browser-based portal built on the MERN stack: MongoDB, Express.js, React and Node.js.

The application supports two authenticated roles: student and placement-cell administrator. Public visitors can browse and search job opportunities. Students can create an account with their roll number, branch and CGPA, sign in, search and filter jobs, apply with a cover note and resume link, track each application through the stages applied, shortlisted, interview, selected or rejected, and withdraw an application while it is still pending. Administrators can publish, edit, close and delete jobs, view every applicant with their academic details, change application status, add remarks, and follow live statistics on a dashboard. The frontend is developed with React 18, Vite, React Router, reusable components and hooks, with responsive light and dark themes.

Authentication uses bcrypt to hash passwords and JSON Web Tokens (JWT) to represent sessions, and middleware checks identity and role before every protected operation. The central business rule, that a student cannot apply twice to the same job, is enforced in three layers: a unique compound index on the student and job references in MongoDB, an explicit check in the API that returns HTTP 409 Conflict, and a user interface that replaces the apply form with the application status. Applications to closed or expired jobs and from students who do not meet the minimum CGPA or branch criteria are rejected by the server. An automated end-to-end API test script of 29 cases, including three simultaneous duplicate requests, passed completely.

GRAPHICAL ABSTRACT

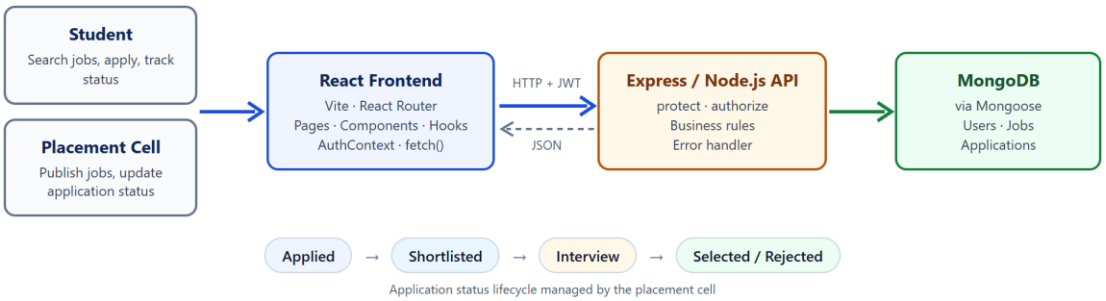


Figure GA.1. Conceptual architecture and application lifecycle of Offerly

Students and the placement cell interact with the React application in a browser. The frontend sends HTTP requests carrying a JWT to the Express/Node.js REST API and receives JSON responses. Middleware authenticates requests and checks roles, route handlers apply the placement business rules, and Mongoose provides the data layer to MongoDB, where users, jobs and applications are stored. Each application then moves through the status lifecycle managed by the placement cell.

ABBREVIATIONS

Table P.1. Abbreviations used in this report

ABBREVIATION	EXPANSION
API	Application Programming Interface
CGPA	Cumulative Grade Point Average
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CTC	Cost to Company (annual package)
DB	Database
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LPA	Lakhs Per Annum
MERN	MongoDB, Express.js, React, Node.js
ODM	Object Document Mapper
REST	Representational State Transfer
SPA	Single-Page Application
UI	User Interface
UID	University Identification Number
URL	Uniform Resource Locator
Vite	Frontend build tool and development server

SYMBOLS AND NOTATION

Table P.2. Symbols and notation

SYMBOL / TERM	MEANING IN THIS REPORT
{ student, job }	Compound key that uniquely identifies one student's application to one job
status	Application stage: applied, shortlisted, interview, selected or rejected
minCgpa	Minimum CGPA required to apply for a job (0 means no cut-off)
eligibleBranches	Branches allowed to apply for a job (empty means all branches)
deadline	Last date on which a job accepts applications
role	User type: student or admin (placement cell)
GET / POST / PUT / DELETE	HTTP methods used to read, create, update and remove resources
201	HTTP status: resource created successfully
400 / 401 / 403	HTTP status: invalid request / not logged in / not permitted for this role
409	HTTP status: conflict, used when a student has already applied to a job

CHAPTER 1
INTRODUCTION

1.1 Client and Need Identification

The intended client is the Training and Placement Cell of a university, together with the final- and pre-final-year students it serves. Every placement season the cell coordinates dozens of company drives for full-time roles and internships. For each drive it must announce the opportunity, state the eligibility criteria, collect applications, shortlist candidates and communicate results. Students, in turn, need to know which companies are hiring, whether they are eligible, and what happened to the applications they submitted.

In the current manual process, opportunities are shared through notice boards, messaging groups and e-mails, and applications are gathered through spreadsheets or online forms. Information is spread across many channels, eligibility is checked by hand, students sometimes submit the same application more than once, and results are communicated individually. These observations define the practical need addressed by this project. No formal institutional survey was carried out; the points are treated as workflow problems identified for the prototype.

1.1.1 Primary users

- **Students:** browse and search jobs, check eligibility, apply, track application status, and withdraw a pending application.
- **Placement-cell administrators:** publish, edit, close and delete jobs, review applicants and update application status with remarks.
- **Public visitors:** view the list of open jobs and job details without logging in.

1.1.2 Need statement

A centralised, database-backed placement portal is needed that gives students one reliable list of opportunities, enforces eligibility and one-application-per-job rules automatically, and gives both students and the placement cell a live, shared view of every application’s status.

1.2 Contemporary Context and Problem Identification

Most student services, from fee payment to examination results, are now delivered through web portals, and students expect the same convenience for placements. A placement portal must, however, protect personal data such as CGPA and contact details, allow each user to perform only the actions permitted for their role, and keep its records consistent even when many students apply at the same moment shortly before a deadline.

The broad problem is the absence of a single, consistent workflow for publishing placement opportunities and managing student applications. It includes fragmented information, manual eligibility checks, duplicate applications, applications after deadlines, and no transparent status tracking.

1.2.1 Problem dimensions

Table 1.1. Problem dimensions and requirements

DIMENSION	OBSERVED WORKFLOW RISK	REQUIREMENT
Job discovery	Students search across separate announcements and may miss a drive.	One job listing with search by company, role or city and filters by type and status.
Eligibility	Placement staff check CGPA and branch manually for every applicant.	Server-side check of minimum CGPA and eligible branches when applying.
Duplicate applications	Repeated form submissions create duplicate entries.	Allow only one application per student per job.
Deadlines	Late applications arrive after a drive has closed.	Reject applications to closed jobs or jobs past their deadline.
Status tracking	Students do not know whether they were shortlisted or rejected.	Status lifecycle visible to the student and editable by the placement cell.
Access control	Anyone with a form link can view or change data.	Login, hashed passwords, JWT and role-based permissions.

1.3 Identification of Tasks

The project was divided into tasks that move from understanding the problem to implementation, validation and documentation. Each task has a deliverable so that the work can be reviewed systematically.

Table 1.2. Project tasks and deliverables

TASK	WORK INCLUDED	DELIVERABLE
T1 – Requirement analysis	Identify roles, user journeys, scope and business rules.	Requirement and scope description.
T2 – Technology selection	Select frontend, backend, database and authentication tools.	Technology stack.
T3 – System design	Define client-server flow, API boundaries and data relationships.	Architecture and database design.
T4 – Backend development	Express routes, middleware, Mongoose models, seed data.	Node.js / Express REST API.
T5 – Frontend development	Reusable components, pages, forms, routing, state handling.	React client.
T6 – Integration	Connect UI actions to REST endpoints and persistent data.	Integrated application.
T7 – UI refinement	Branding (Offerly logo), responsive layout, dark mode.	Polished interface.
T8 – Validation	Automated API tests and browser checks of core workflows.	Test script and results.
T9 – Documentation	README, user manual, project report.	Final project report and repository.

1.4 Project Objectives and Scope

1.4.1 Objectives

1. Develop a responsive React interface using reusable components and JSX.
2. Use React state and effects to manage forms, retrieve data and update the interface.
3. Implement page navigation and protected, role-based routes with React Router.
4. Build a Node.js backend that exposes GET, POST, PUT and DELETE endpoints with Express.js.
5. Implement registration and login using bcrypt password hashing and JWT.
6. Model users, jobs and applications with Mongoose schemas, references, validation and .populate().
7. Integrate the frontend and backend using fetch() with database-backed CRUD operations.
8. Enforce the one-application-per-job rule along with deadline and eligibility rules.

1.4.2 In-scope functionality

- Public job listing, job details, keyword search and filters.
- Student registration with roll number, branch and CGPA; login; profile editing.

- Applying with a cover note and resume link; application history; withdrawal while pending.
- Administrator job creation, editing, closing and deletion with automatic removal of related applications.
- Administrator view of all applicants, status updates, remarks and dashboard statistics.
- JWT authentication, role checks, ownership checks and MongoDB persistence with integrity constraints.
- Responsive layout for phones and desktops with a light/dark theme toggle.

1.4.3 Out-of-scope functionality

The prototype does not include resume file uploads, automated e-mail or SMS notifications, interview scheduling, recruiter (company) accounts, offer-letter management or analytics exports. These are listed as future work and are not represented as implemented features.

1.5 Timeline

The project followed the staged workflow shown below. The chart shows the order and overlap of activities rather than exact calendar durations.

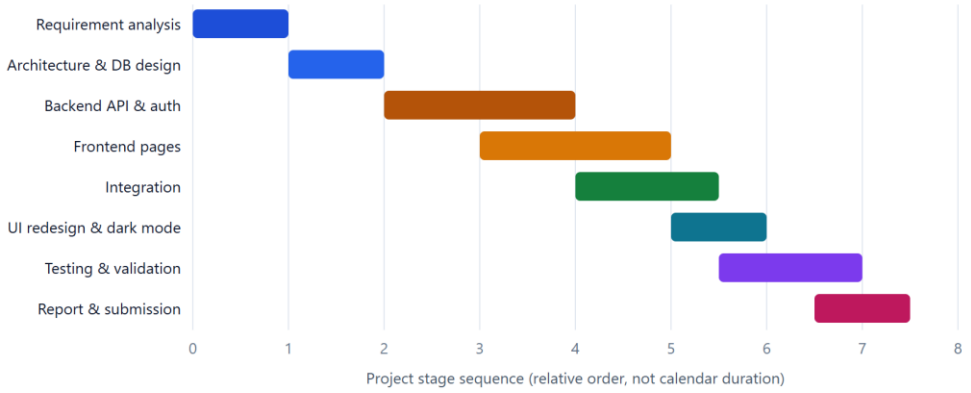


Figure 1.1. Sequence of project development activities

1.6 Organization of the Report

Chapter 1 introduces the client, need, problem, tasks, objectives, scope and timeline. Chapter 2 describes concept generation, feature evaluation, design constraints, alternative designs, the selected design, system architecture, database design and implementation methodology. Chapter 3 explains the implementation of the frontend, backend,

authentication and business rules, and presents the validation results and the mapping to the nine laboratory experiments. Chapter 4 presents the conclusion, limitations and future work. The appendices contain the user manual, screenshots of the running application, and the source code structure.

CHAPTER 2
DESIGN FLOW / PROCESS

2.1 Concept Generation

The design began by identifying the entities and actions in a placement workflow. The essential entities are users, jobs and applications. A user is either a student or a placement-cell administrator. Administrators post jobs, and students create applications that link them to jobs. Each application carries a status that the placement cell advances as the recruitment process progresses. This conceptual model is the basis of the database schema and the API boundaries.

2.1.1 Core use cases

Table 2.1. Core use cases

ACTOR	USE CASE	EXPECTED SYSTEM RESPONSE
Public visitor	Browse and search jobs	Return jobs matching the search text, type and status filters.
Student	Register / sign in	Validate input, hash the password, issue a JWT.
Student	Apply to a job	Check role, deadline, eligibility and duplicates; create the application.
Student	Track applications	Return own applications with job details and current status.
Student	Withdraw application	Delete own application only while its status is “applied”.
Admin	Publish / edit / close job	Validate and save the job with the admin as poster.
Admin	Delete job	Delete the job and all of its applications.
Admin	Review applicants	Return applications with student and job details (populated).
Admin	Update status / remark	Validate the status value and save it; the student sees it immediately.

2.2 Evaluation and Selection of Specifications / Features

Features were evaluated by their contribution to the core problem, implementation complexity, importance for data integrity, and suitability for a laboratory mini project. The selected set covers the complete placement lifecycle and its most important rules.

Table 2.2. Candidate features and evaluation

FEATURE	VALUE TO USER	EFFORT	DECISION
Public job listing and search	Students find opportunities quickly.	Low	Include
Type and status filters	Separates internships, full-time and closed drives.	Low	Include
Student and admin authentication	Identifies users and protects data.	Medium	Include
Eligibility check (CGPA, branch)	Removes manual screening work.	Medium	Include
Duplicate-application prevention	Keeps one clean record per applicant.	Medium	Include
Deadline and closed-job checks	Stops late applications.	Low	Include
Status lifecycle with remarks	Transparent progress for students.	Medium	Include
Admin dashboard statistics	Quick overview of the season.	Low	Include
Dark mode and responsive layout	Comfortable use on any device.	Low	Include
Resume file upload	Convenient, but needs file storage.	Medium	Future scope
E-mail notifications	Instant alerts; needs a mail service.	Medium	Future scope
Recruiter accounts	Companies review applicants directly.	High	Future scope

2.3 Design Constraints

The solution was designed under technical, economic, security, ethical, social and professional constraints, which determined the practical feature set and how the system should be used.

Table 2.3. Design constraints and responses

CONSTRAINT AREA	CONSIDERATION	DESIGN RESPONSE
Technical	Must run on a standard Node.js and browser setup.	React with Vite, Express, MongoDB and Mongoose.
Economic	A student project should not need paid infrastructure.	Free local tools; an in-memory MongoDB mode for demos; free hosting tiers for deployment.
Security	Passwords and protected actions need safeguards.	bcrypt hashing, JWT verification, role checks, secrets kept in .env.
Privacy and ethics	CGPA and contact details are personal data.	Applicant details visible only to administrators; passwords never returned by the API.
Data integrity	Many students may apply at the same moment.	Unique compound index in the database in addition to API checks.
Social / usability	Users have different devices and preferences.	Responsive layout, clear feedback messages, light and dark themes.
Deployment	The project must be reproducible on a student laptop.	Documented setup, seed script with demo accounts, single-service production build.
Professional	The report must separate implemented and proposed work.	Limitations and future work are listed explicitly.

2.4 Analysis and Feature Finalization Subject to Constraints

After applying the constraints, the project keeps every feature that directly supports job discovery, applying, eligibility, status tracking and job management. Features that need external services (file storage, e-mail) or a third role (recruiters) are deferred. Role-based access separates student and administrator responsibilities, and database-level uniqueness is used because application-only checks are not sufficient when requests arrive simultaneously.

2.5 Alternative Design Approaches

2.5.1 Alternative A – Single-page application with REST backend

A React single-page application communicates with a separate Express REST API. The backend owns authentication, business rules and persistence, while the frontend handles presentation, navigation, form state and feedback. This is the approach selected for Offerly.

2.5.2 Alternative B – Server-rendered application

Pages could instead be rendered on the server with a template engine, combining views with request handling. This is simpler for small sites but would not demonstrate React components, hooks, routing and frontend-backend integration as directly, and it would not produce a reusable API.

Table 2.4. Alternative architecture comparison

CRITERION	A: REACT + REST API	B: SERVER-RENDERED
Separation of concerns	Clear client / API boundary.	Views and server logic tightly coupled.
Coverage of lab experiments	Demonstrates components, hooks and routing directly.	Limited React coverage.
API reuse	The same API can serve a mobile app later.	Would need a separate API.
User experience	Fast in-page updates without reloads.	Full page reload on most actions.
Initial complexity	Two projects to set up.	One project.
Fit with project objectives	High.	Moderate.

2.6 Design Selection

Alternative A was selected because the project brief emphasises React components, hooks, React Router, Node.js, Express REST APIs, JWT and bcrypt authentication, MongoDB with Mongoose, and frontend-backend integration. A separate React client and Express API demonstrate all of these directly, and the API boundary allows future clients such as a mobile application to reuse the backend. In production, Express also serves the built React files, so the system can still be deployed as a single web service.

2.7 System Architecture and Data Flow

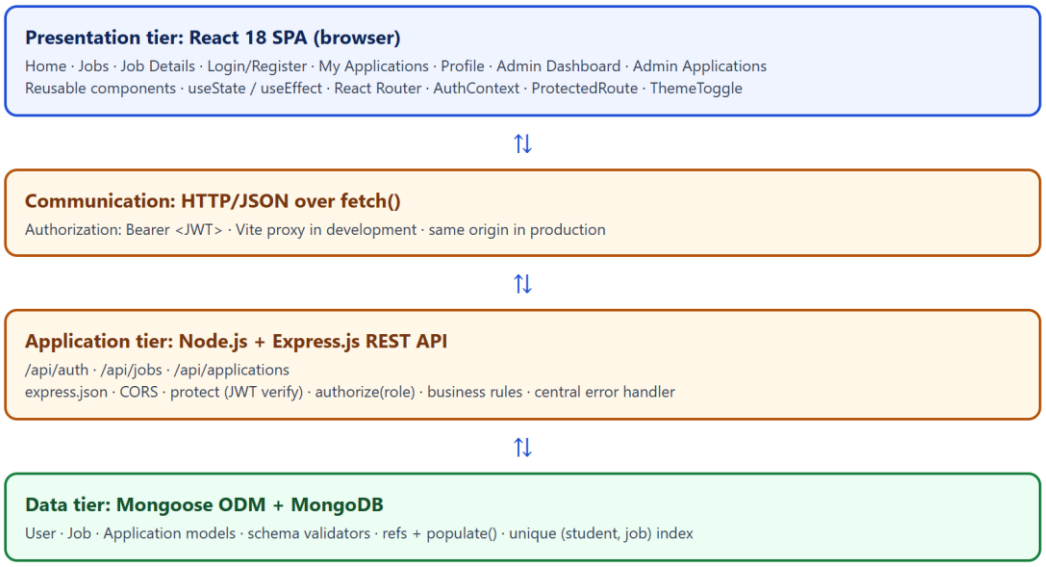


Figure 2.1. Three-tier architecture of Offerly

The architecture follows a client-server model. React renders the interface in the browser and sends HTTP requests to the Express API. Express middleware parses JSON, verifies the JWT on protected routes, checks the user's role, and passes the request to a route handler. Handlers apply the business rules and use Mongoose models to validate and query MongoDB. The API responds with JSON, and the React components update their state to reflect the result.

2.7.1 Application sequence

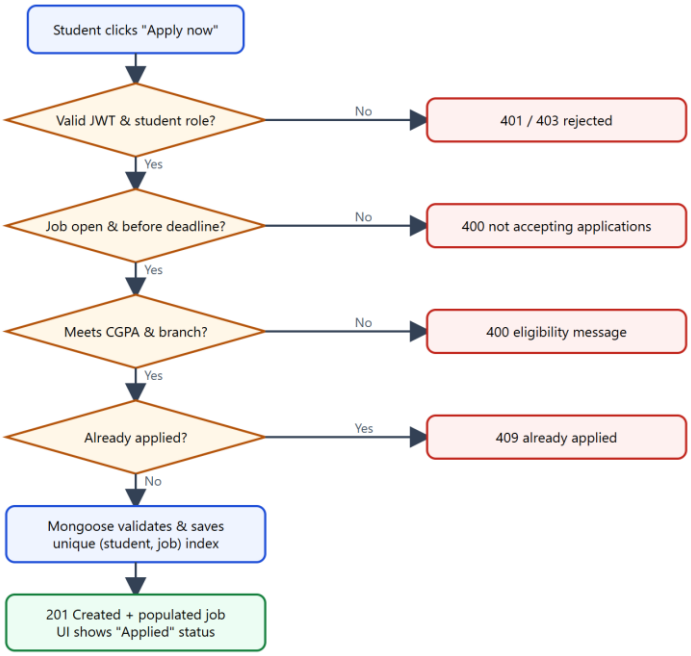


Figure 2.2. Process flow when a student applies for a job

1. The student opens a job and clicks “Apply now”.
2. React sends POST /api/applications with the job id, cover note and resume link, and the JWT in the Authorization header.
3. The protect middleware verifies the token and loads the user; authorize('student') confirms the role.
4. The handler checks that the job exists, is open and is not past its deadline.
5. The handler checks the student’s CGPA against the job’s minimum and the student’s branch against the eligible branches.
6. The handler checks for an existing application by the same student for the same job and returns 409 if one exists.
7. Mongoose validates and saves the application; the unique (student, job) index rejects any duplicate created by a simultaneous request.
8. The API returns 201 with the job populated, and React replaces the form with the application status.

2.8 Database Design

MongoDB stores the data in three collections: users, jobs and applications. Mongoose schemas define structure, validation rules, references and indexes. References link records without copying the full user or job into every application, and .populate() resolves them when the data is read.

2.8.1 User collection

Table 2.5. User collection fields

FIELD	PURPOSE / RULE
name	Display name; required, 2–60 characters.
email	Unique, stored in lowercase, validated by a regular expression.
password	bcrypt hash; minimum 6 characters before hashing; excluded from queries (select: false).
role	student (default) or admin; admin requires the placement-cell code at registration.
rollNo, branch, cgpa	Student profile; branch from a fixed list; CGPA between 0 and 10.
createdAt, updatedAt	Automatic timestamps.

2.8.2 Job collection

Table 2.6. Job collection fields

FIELD	PURPOSE / RULE
company, title, description	Required text fields with maximum lengths.
type	Full-time or Internship (enum).
ctc	Package in LPA (full-time) or stipend in thousand per month (internship); cannot be negative.
location	Job location; defaults to On-site.
minCgpa	Minimum CGPA, 0–10; 0 means no cut-off.
eligibleBranches	List of allowed branches; empty means all branches.
deadline, status	Last date to apply; open or closed.
postedBy	Reference to the administrator (User) who published the job.

2.8.3 Application collection

Table 2.7. Application collection fields

FIELD	PURPOSE / RULE
student	Reference to the student User; required.
job	Reference to the Job; required.
status	applied, shortlisted, interview, selected or rejected (enum); default applied.
coverNote	Optional, up to 500 characters.
resumeLink	Optional; must be a valid http(s) URL.
adminRemark	Optional note from the placement cell, up to 300 characters.
{ student, job } index	Unique compound index: one application per student per job.

2.8.4 Entity relationships

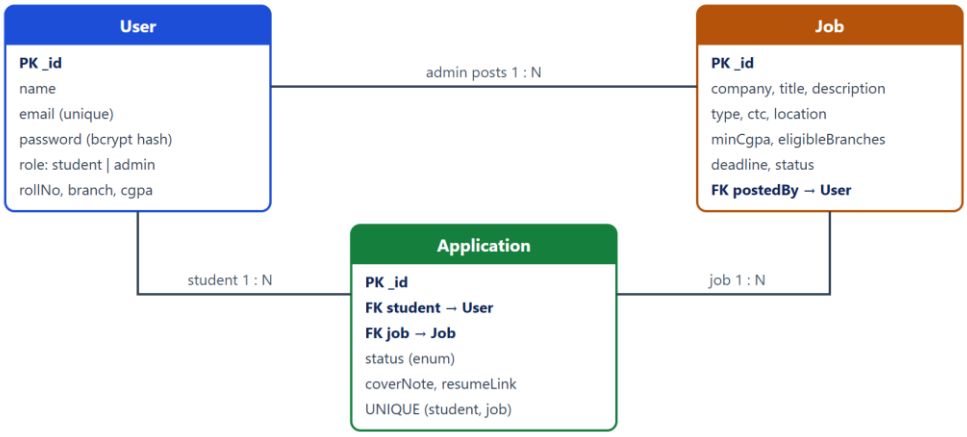


Figure 2.3. Entity relationship overview

Table 2.8. Entity relationships

RELATIONSHIP	CARDINALITY	DESCRIPTION
Admin (User) → Job	One-to-many	An administrator may post many jobs; each job stores one postedBy reference.
Student (User) → Application	One-to-many	A student may apply to many different jobs.
Job → Application	One-to-many	A job may receive many applications.
Student + Job → Application	At most one	The unique index allows only one application for each student–job pair.

2.9 Implementation Methodology

Implementation was divided into backend, frontend, integration and refinement activities. The backend was built first: the database connection, Mongoose models, authentication routes and middleware, then job and application routes with a central error handler and a

seed script for demo data. The frontend was then developed with an API wrapper, an authentication context, reusable components and page components. Integration connected each user action to its REST endpoint and confirmed that the interface reflected the stored data. Finally the interface was refined with the Offerly branding, a responsive layout and dark mode, and the system was validated with an automated test script and checks in the browser.

CHAPTER 3
RESULTS ANALYSIS AND VALIDATION

3.1 Implementation Environment and Modern Engineering Tools

JavaScript is used on both client and server. React 18 and Vite support frontend development, and React Router manages navigation. Node.js and Express.js provide the API runtime and routing. MongoDB is the document database and Mongoose provides schema-based modelling. bcryptjs and jsonwebtoken provide password hashing and signed tokens. Git and GitHub are used for version control, and npm scripts install dependencies, seed the database and run the servers.

Table 3.1. Implementation environment

LAYER	TOOLS / LIBRARIES	RESPONSIBILITY
Frontend	React 18, Vite, React Router 6, CSS	Components, pages, forms, routing, state, themes.
Backend	Node.js, Express.js 4, CORS, dotenv	HTTP server, middleware, routes, business rules.
Persistence	MongoDB, Mongoose 8	Storage, schema validation, references, indexes.
Authentication	bcryptjs, jsonwebtoken	Password hashing; token creation and verification.
Integration	fetch(), JSON, Vite proxy	Communication between browser and API.
Development	nodemon, mongodb-memory-server, Git, GitHub	Auto-restart, in-memory demo database, version control.

3.2 Frontend Implementation

The frontend is organised into reusable components and page components. Navbar, JobCard, CompanyAvatar, StatusBadge, StatCard, JobForm, Alert, Loader, EmptyState, AuthLayout, Logo and ThemeToggle are shared across pages, which keeps the interface consistent and avoids duplicated markup.

3.2.1 React components and JSX

JSX describes the interface with markup-like syntax inside JavaScript. JobCard receives a job object through props and renders the company badge, role, package, eligibility chips, deadline and applicant count; the Home and Jobs pages render lists of JobCards with

map(). StatusBadge maps an application status to a coloured label, so one change to the component updates every table that uses it.

3.2.2 State and effects

useState manages form inputs, lists, loading flags, error and success messages, and filters. useEffect fetches jobs, applications and statistics when a page opens or a filter changes, and restores the logged-in user from the saved token when the application starts. After a create, update or delete request succeeds, the component updates its state directly, so the change appears immediately without reloading the page.

3.2.3 Routing and protected pages

React Router maps URLs to pages: Home, Jobs, Job Details (/jobs/:id), Login, Register, My Applications, Profile, Admin Dashboard and Admin Applications. The ProtectedRoute component redirects visitors who are not logged in to the login page and redirects users with the wrong role to their own home page. Search filters are stored in the URL with useSearchParams, so a filtered list can be shared. Backend authorisation is still enforced, because frontend route protection alone cannot secure the API.

3.2.4 Search, filters, feedback and themes

The jobs page offers a debounced search box and filters for job type and status. Conditional rendering shows loading spinners, empty states, validation messages and API errors, and the apply panel changes according to who is viewing: a login prompt for visitors, the application form for eligible students, the current status for students who have applied, and an applicant count for administrators. A theme toggle switches between light and dark modes and remembers the choice, and the layout adapts to phone screens by stacking tables into cards.

3.3 Backend Implementation

The backend starts from server/src/server.js, which loads environment variables, connects to MongoDB and starts Express. app.js configures CORS, JSON parsing, request logging and the route modules, and, in production, serves the built React application. A central error handler converts Mongoose validation errors, invalid ids and duplicate-key errors into clear JSON messages with the correct HTTP status codes.

3.3.1 Express route organisation

Authentication routes handle registration, login and the current user's profile. Job routes provide the public listing with search and filters, job details with applicant counts, and administrator create, update and delete operations. Application routes handle applying, a student's own applications, the administrator's full list with filters, statistics, status updates and withdrawal. Each module is small and focused, which makes the API easy to inspect.

3.3.2 REST API summary

Table 3.2. REST API summary

METHOD AND ENDPOINT	ACCESS	PURPOSE
POST /api/auth/register	Public	Create a student account (admin with valid code).
POST /api/auth/login	Public	Verify credentials and issue a JWT.
GET /api/auth/me	Logged in	Return the current user's profile.
PUT /api/auth/me	Logged in	Update name, roll number, branch, CGPA or password.
GET /api/jobs?search=&type=&status=	Public	List jobs with search and filters, including applicant counts.
GET /api/jobs/:id	Public	Return one job.
POST /api/jobs	Admin	Create a job.
PUT /api/jobs/:id	Admin	Update a job.
DELETE /api/jobs/:id	Admin	Delete a job and its applications.
POST /api/applications	Student	Apply to a job; duplicate returns 409.
GET /api/applications/me	Student	List own applications with job details.
GET /api/applications?job=&status=	Admin	List all applications with student and job details.
GET /api/applications/stats	Admin	Counts for the dashboard.
PUT /api/applications/:id	Owner / Admin	Admin: status and remark. Student: edit while applied.
DELETE /api/applications/:id	Owner / Admin	Withdraw (student, only while applied) or remove.

3.4 Authentication and Authorization

When an account is created, a Mongoose pre-save hook hashes the password with bcrypt (10 salt rounds) before the user is stored. At login the submitted password is compared with the stored hash, and a successful login returns a JWT signed with a secret held in the server environment. The React client stores the token and sends it in the Authorization header of every protected request.

The protect middleware verifies the token and loads the user; the authorize middleware restricts a route to one or more roles. Ownership is checked for applications: a student can view, edit or withdraw only their own applications, while administrators can manage all of them. Registering as an administrator requires the placement-cell code configured on the server.

3.4.1 Security considerations

- Only password hashes are stored, and the password field is excluded from every query result.
- The JWT secret, database URI and admin code are kept in a private .env file that is excluded from Git.
- Every request body is validated on the server by Mongoose schemas, even when the frontend also validates it.
- Role and ownership checks are applied to every protected route.
- Regular-expression special characters in search text are escaped before querying the database.
- Production deployment should use HTTPS; no passwords or live tokens appear in the screenshots in this report.

3.5 Application Integrity and Business Rules

3.5.1 Duplicate-application prevention

Before creating an application, the API checks whether the student has already applied to the job and returns HTTP 409 Conflict with the message “You have already applied to this job”. An API check alone can be bypassed by two requests that arrive at the same moment, because both may pass the check before either is saved. The unique compound index on { student, job } therefore provides the final guarantee: MongoDB rejects the second insert, and the error handler converts the duplicate-key error into the same 409 response.

3.5.2 Deadline and eligibility enforcement

A job accepts applications only while its status is open and until the end of its deadline date. If the job has a minimum CGPA, the student must have a CGPA in their profile that meets it; if the job lists eligible branches, the student’s branch must be one of them. Each

failure returns HTTP 400 with a specific message, for example “Minimum CGPA for this job is 8; yours is 6.8”.

3.5.3 Withdrawal, status updates and job deletion

A student may edit or withdraw an application only while its status is “applied”; once the placement cell has shortlisted it, the application is locked. Status values are validated against the allowed list. When a job is deleted, all of its applications are deleted as well, so no orphaned records remain.

3.5.4 Business-rule summary

Table 3.3. Business rules and enforcement

RULE	ENFORCEMENT LOCATION	EXPECTED OUTCOME
One application per student per job	API check + unique compound index	Duplicate rejected with 409.
No applications to closed or expired jobs	Job.isAcceptingApplications()	Rejected with 400.
Minimum CGPA and eligible branches	Application route eligibility checks	Rejected with a specific message.
Only students apply; only admins manage jobs	authorize() middleware	Rejected with 403.
Students act only on their own applications	Ownership check	Rejected with 403.
Withdraw only while “applied”	Status check before delete	Rejected with 400 after shortlisting.
Deleting a job removes its applications	Job delete route	No orphaned applications.

3.6 API and Database Validation

The application was validated with an automated end-to-end test script that sends real HTTP requests to the running API backed by MongoDB with seeded demo data. The script contains 29 test cases covering registration, login, validation errors, protected routes, role checks, search and filters, full CRUD on jobs and applications, .populate() results, cascading deletion and every business rule. All 29 cases passed (Figure 3.1). The main flows were also checked manually in the browser, including applying as a student, updating that application’s status as the administrator and seeing the new status on the student’s page.

Table 3.4. Validation test cases and observed results

ID	SCENARIO	EXPECTED RESULT	OBSERVED
TC-01	Log in with correct admin credentials.	JWT returned.	Passed
TC-02	Log in with a wrong password.	401 Unauthorized.	Passed
TC-03	Register a new student.	201; user returned without password.	Passed
TC-04	Register with an existing e-mail.	409 Conflict.	Passed
TC-05	Register with invalid fields.	400 with field errors.	Passed
TC-06	Register as admin with a wrong code.	400 Bad Request.	Passed
TC-07	Open a protected route without a token.	401 Unauthorized.	Passed
TC-08	Search jobs for “micro”.	Only the Microsoft job returned.	Passed
TC-09	Filter jobs by type Internship.	Only internships returned.	Passed
TC-10	Student tries to create a job.	403 Forbidden.	Passed
TC-11	Admin creates and updates a job.	201, then 200 with new values.	Passed
TC-12	Create a job with a negative CTC.	400 validation error.	Passed
TC-13	Student applies to an eligible job.	201 with populated job.	Passed
TC-14	Student applies to the same job again.	409 already applied.	Passed
TC-15	Three identical apply requests at once.	Exactly one 201.	Passed
TC-16	Apply to a closed job.	400 Bad Request.	Passed
TC-17	Student below minimum CGPA applies.	400 eligibility message.	Passed
TC-18	Student lists own applications.	Only own applications returned.	Passed
TC-19	Student lists all applications.	403 Forbidden.	Passed
TC-20	Admin lists applications.	Student and job populated.	Passed
TC-21	Admin sets status to shortlisted.	200 with new status.	Passed
TC-22	Admin sets an invalid status.	400 validation error.	Passed
TC-23	Student withdraws after shortlisting.	400 Bad Request.	Passed
TC-24	Another student deletes the application.	403 Forbidden.	Passed
TC-25	Admin deletes a job.	Job and its applications removed.	Passed
TC-26	Request a job with an invalid id.	400 Bad Request.	Passed



Figure 3.1. Output of the automated API test script (29 of 29 passed)

3.7 Results Interpretation

The results show that Offerly covers the complete placement workflow, from public job discovery through student applications to administrator decisions, and that its rules hold under both normal and invalid use. Unauthorised and wrongly-roled requests were rejected with 401 and 403, invalid data was rejected by schema validation, and every business rule produced the expected response. In particular, when three identical apply requests were sent at the same moment, exactly one application was created, which confirms that the database-level unique index protects the rule under concurrency where an API-only check would not.

In the browser, an application submitted by a student appeared immediately in the administrator's list, and a status change made by the administrator appeared immediately on the student's My Applications page. The interface also worked at phone width without horizontal scrolling. Performance and usability were not measured formally, so they are described as observed behaviour rather than measured results.

3.8 Mapping to the Nine Lab Experiments

Table 3.5. Mapping of project work to the nine lab experiments

EXPERIMENT	PROJECT IMPLEMENTATION / EVIDENCE
1. Reusable components and JSX	Navbar, JobCard, CompanyAvatar, StatusBadge, StatCard, JobForm, Alert, Loader; lists rendered with map().
2. React hooks	useState for all controlled forms and UI state; useEffect for fetching jobs, applications and statistics and for restoring the session.
3. React Router	Nine routes in App.jsx; useParams for /jobs/:id; useSearchParams for filters; ProtectedRoute for student and admin pages.
4. Interactive frontend	Search box, type and status filters, status pills, cards and tables, role-based conditional rendering, dark mode.
5. Node.js	server.js starts the backend, loads .env and connects to MongoDB; seed.js loads demo data.
6. Express REST API	GET, POST, PUT and DELETE routes for auth, jobs and applications with a central error handler.
7. JWT and bcrypt	Pre-save bcrypt hashing, JWT issued at login, protect and authorize middleware, ProtectedRoute in React.
8. MongoDB and Mongoose	Users, Jobs and Applications with validators, references, .populate() and a unique compound index.
9. Frontend–backend integration	api/client.js wraps fetch() with the JWT; every create, update and delete is reflected in React state.

CHAPTER 4
CONCLUSION AND FUTURE WORK

4.1 Conclusion

Offerly demonstrates the design and implementation of a complete full-stack web application with MongoDB, Express.js, React and Node.js. It replaces scattered placement announcements and spreadsheets with one portal in which the placement cell publishes jobs and manages applications, and students discover opportunities, apply and follow their progress.

The project also demonstrates key backend engineering concepts. Passwords are hashed with bcrypt, requests are authenticated with JWTs, role checks separate student and administrator operations, and ownership checks protect each student's applications. Mongoose models represent the three related collections with validation, references and .populate(). The rule that a student cannot apply twice to the same job is enforced by both the API and a unique compound index, and it held even under simultaneous requests. All 29 automated tests passed, and every concept from the nine laboratory experiments is applied in the working application.

4.2 Limitations and Deviations from Expected Results

The prototype intentionally keeps a simple role model of student and administrator; administrator accounts are created with a shared placement-cell code, which a production system should replace with institution-managed accounts. The JWT is stored in the browser's localStorage, a simplified choice that is not the strongest production session design. Students provide a resume link rather than uploading a file, and there are no e-mail notifications.

For demonstrations without a MongoDB installation, the project includes an in-memory database mode whose data resets on restart; persistent storage requires a local MongoDB server or MongoDB Atlas. Validation was done with an automated API integration script and manual browser checks; there are no unit tests or automated browser tests yet. No measured response times or user-satisfaction scores are reported. No deviations from the expected functional results were observed in the executed tests.

4.3 Future Work

1. Upload resumes to cloud storage instead of using a resume link.
2. Send e-mail and in-app notifications when an application's status changes.
3. Schedule interviews for shortlisted students with calendar invitations.
4. Add recruiter accounts so companies can post jobs and review applicants directly.
5. Provide analytics such as branch-wise and company-wise placement statistics, with export to Excel or CSV.
6. Add pagination and saved searches for large job lists.
7. Add unit, integration and end-to-end tests in a continuous-integration pipeline.
8. Strengthen production security with HTTPS, secure cookies, rate limiting, e-mail verification, password reset and audit logs.
9. Deploy the application on a cloud platform with MongoDB Atlas and gather feedback from real students and placement staff.

4.4 Way Ahead

The immediate next step is to deploy Offerly as a single web service with a MongoDB Atlas database, so that it can be reached through a public URL. The application can then be piloted with a small group of students and the placement cell during an actual drive, and their feedback used to prioritise the future work listed above.

REFERENCES

1. Express.js. (n.d.). Express – Node.js web application framework. https://expressjs.com/

2. MongoDB, Inc. (n.d.). MongoDB Manual. https://www.mongodb.com/docs/manual/

3. Mongoose. (n.d.). Mongoose documentation. https://mongoosejs.com/docs/

4. Node.js. (n.d.). Node.js documentation. https://nodejs.org/docs/

5. OpenJS Foundation. (n.d.). npm documentation. https://docs.npmjs.com/

6. React. (n.d.). React documentation. https://react.dev/

7. React Router. (n.d.). React Router documentation. https://reactrouter.com/

8. Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT), RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519

9. Vite. (n.d.). Vite documentation. https://vite.dev/guide/

10. Chandigarh University. (2026). Full Stack Development – II (24BET_TPP-603), Experiment 10: Lab-Based Mini Project.

APPENDIX 1 – USER MANUAL

This appendix explains how to install, run and use Offerly on a local computer. The source code is available at https://github.com/Yash1911ind/offerly.

A1.1 Prerequisites

- A computer with Windows, macOS or Linux and internet access for installing dependencies.
- Node.js version 18 or later, with npm.
- Optional: MongoDB Community Server running locally, or a MongoDB Atlas connection string. Without MongoDB, the in-memory demo mode can be used.
- Git, to clone the repository.

A1.2 Installation

1. Clone the repository: git clone https://github.com/Yash1911ind/offerly.git
2. Open a terminal in the offerly folder and install all dependencies: npm run install:all
3. Copy server/.env.example to server/.env.
4. In server/.env, set MONGO_URI to the MongoDB connection string and JWT_SECRET to a long random value. Keep this file private.

A1.3 Running the application

1. **With MongoDB:** load the demo data with npm run seed, then start the API with npm run server.
2. **Without MongoDB:** start the API with npm run server:memory, which creates and seeds an in-memory database.
3. In a second terminal, start the frontend with npm run client.
4. Open http://localhost:5173 in a browser.

Table A1.1. Demo accounts created by the seed data

ROLE	E-MAIL	PASSWORD
Placement cell (admin)	admin@cu.edu	admin123
Student (CSE, CGPA 8.6)	aarav@cu.edu	student123
Student (ECE, CGPA 7.4)	diya@cu.edu	student123
Student (ME, CGPA 6.8)	kabir@cu.edu	student123

A1.4 Student workflow

1. Create a student account on the Join now page with name, e-mail, password, roll number, branch and CGPA, or log in.
2. Open Jobs and use the search box or the type and status filters to find opportunities.
3. Open a job to see its package, minimum CGPA, eligible branches, deadline and description.
4. Optionally write a short note and add a resume link, then click Apply now.
5. Open My Applications to see every application and its current status, and use the status pills to filter.
6. Click Withdraw to cancel an application that has not yet been shortlisted.
7. Use Profile to update CGPA, branch or password, and the moon/sun button to switch themes.

A1.5 Placement-cell workflow

1. Log in with an administrator account to open the Placement dashboard.
2. Click + New job, fill in the job details, eligibility and deadline, and click Publish job.
3. Use Edit to change a job or set its status to Closed, and Delete to remove a job and its applications.
4. Open Applications to see every applicant with roll number, branch, CGPA and cover note; filter by job or status.
5. Change an application’s status from the drop-down and use Remark to add a note for the student.

A1.6 Troubleshooting

Table A1.2. Troubleshooting

ISSUE	CHECK / ACTION
API cannot connect to MongoDB	Start MongoDB or correct MONGO_URI; or use npm run server:memory.
"JWT_SECRET is missing"	Create server/.env from .env.example and set JWT_SECRET.
Frontend shows "Cannot reach the server"	Make sure the API is running on port 5000.
Session expired message	Log in again; tokens expire after the configured period (7 days by default).
Apply button rejected	Check that you are logged in as a student, the job is open, your CGPA and branch are eligible, and you have not already applied.
Port already in use	Stop the other process or change PORT in server/.env.

A1.7 Safe operation

Never share the .env file, database credentials, JWT secret, passwords or live tokens. Use the demo accounts and sample data when preparing screenshots, and hide personal details of real students unless their disclosure is authorised.

APPENDIX 2 – SCREENSHOT EVIDENCE

The following screenshots were captured from the running application, using the seeded demo data.

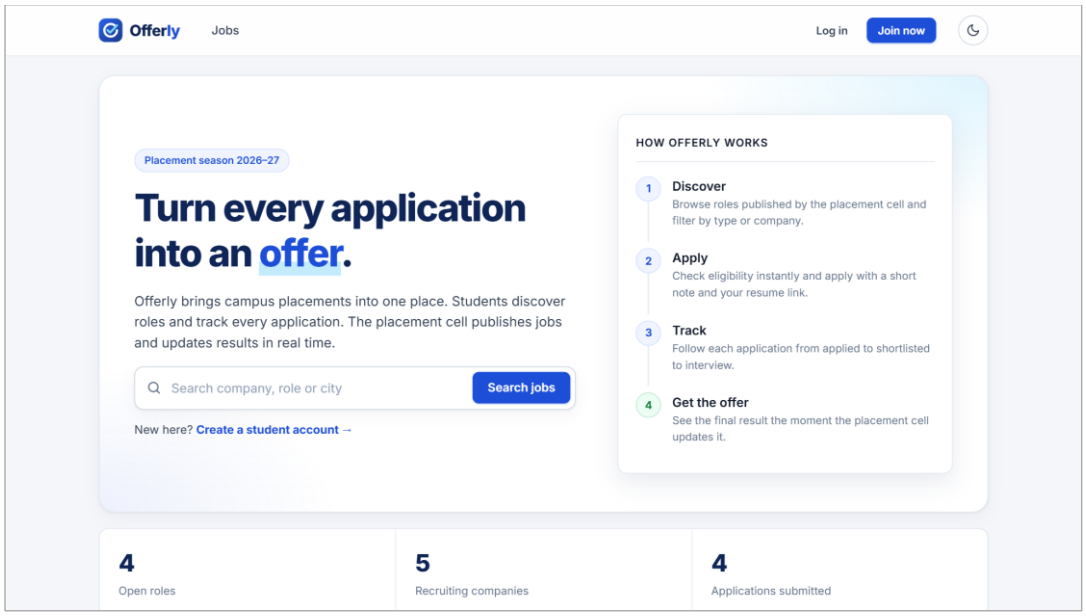


Figure A2.1. Home page with search and live placement statistics

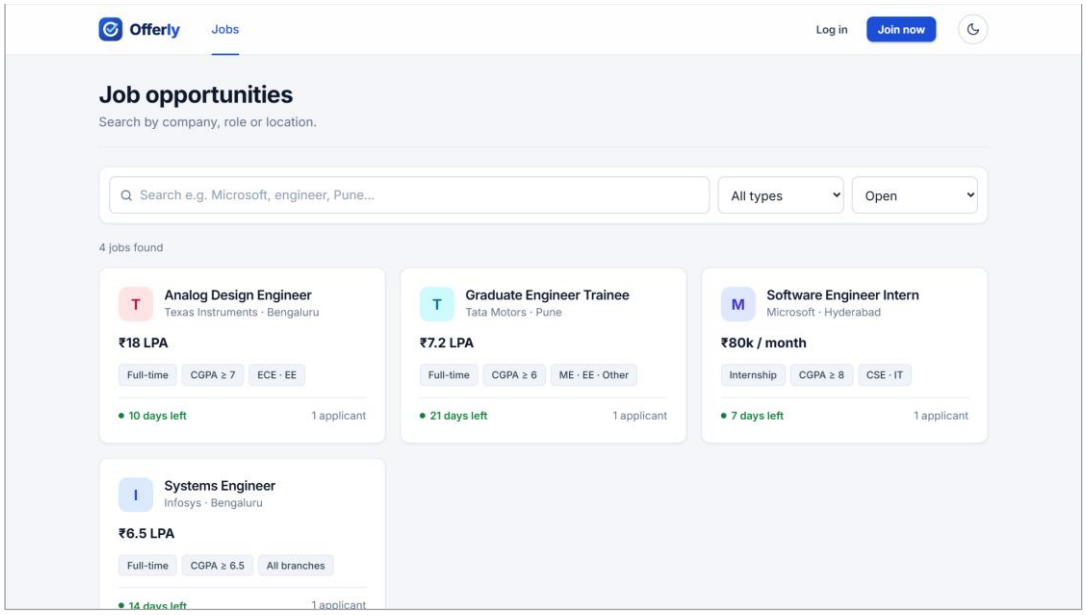


Figure A2.2. Job listing with search and filters

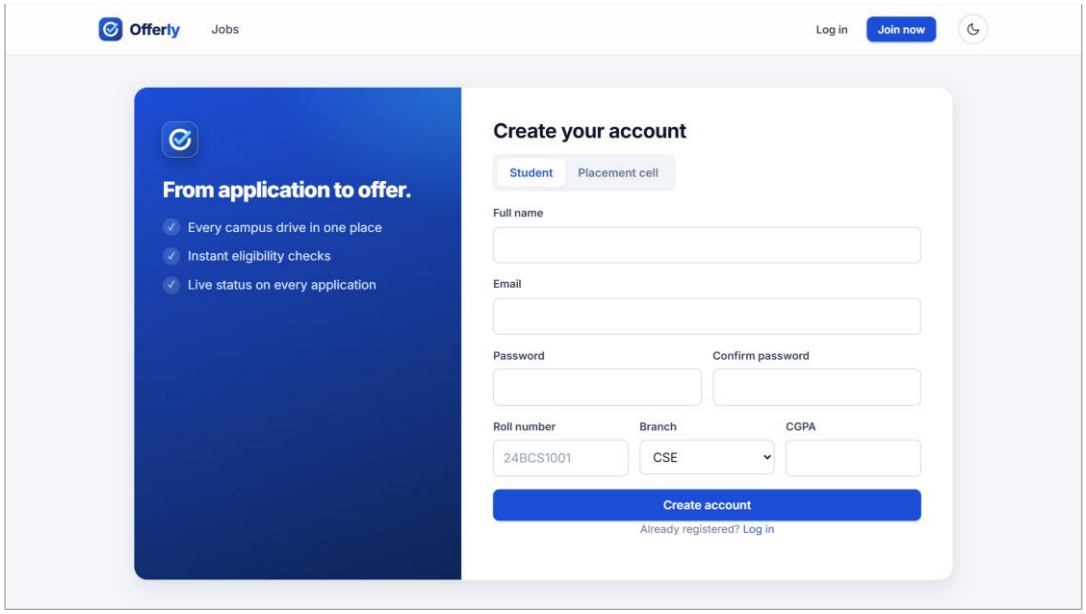


Figure A2.3. Registration page for students and the placement cell

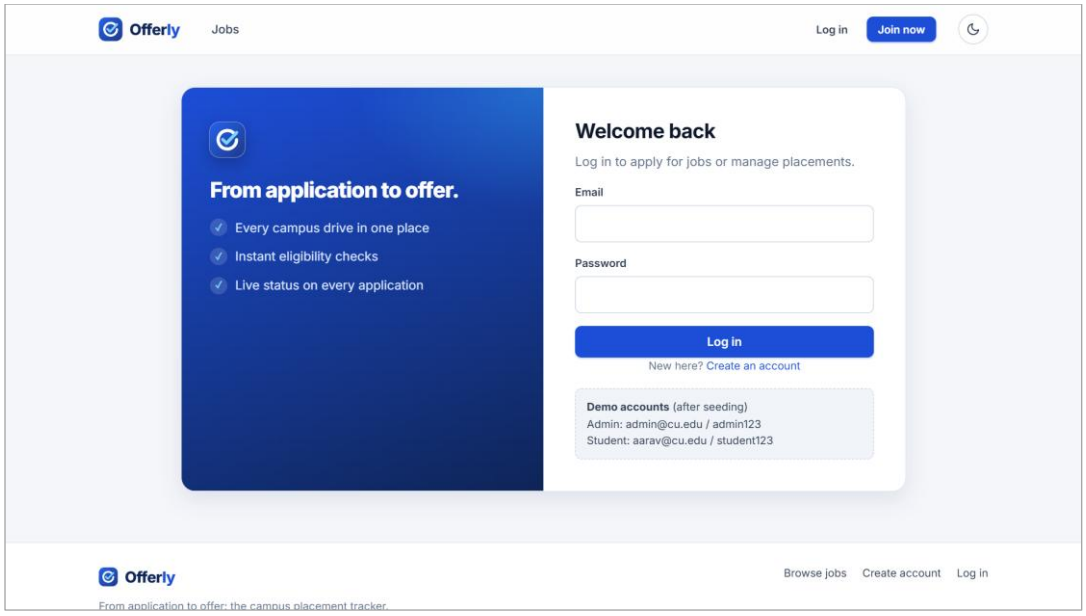


Figure A2.4. Login page

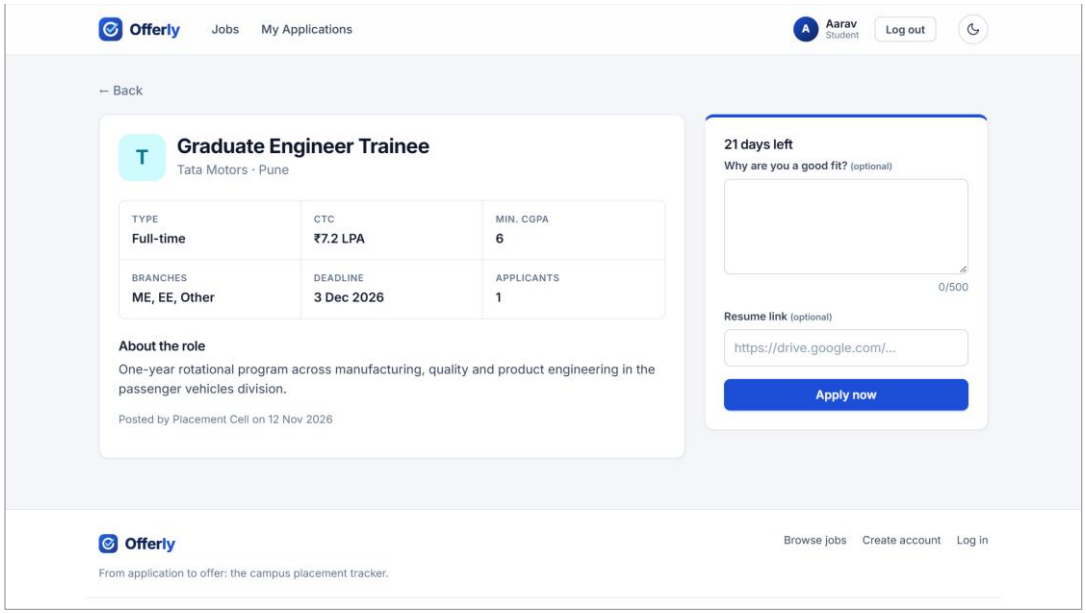


Figure A2.5. Job details with the application form

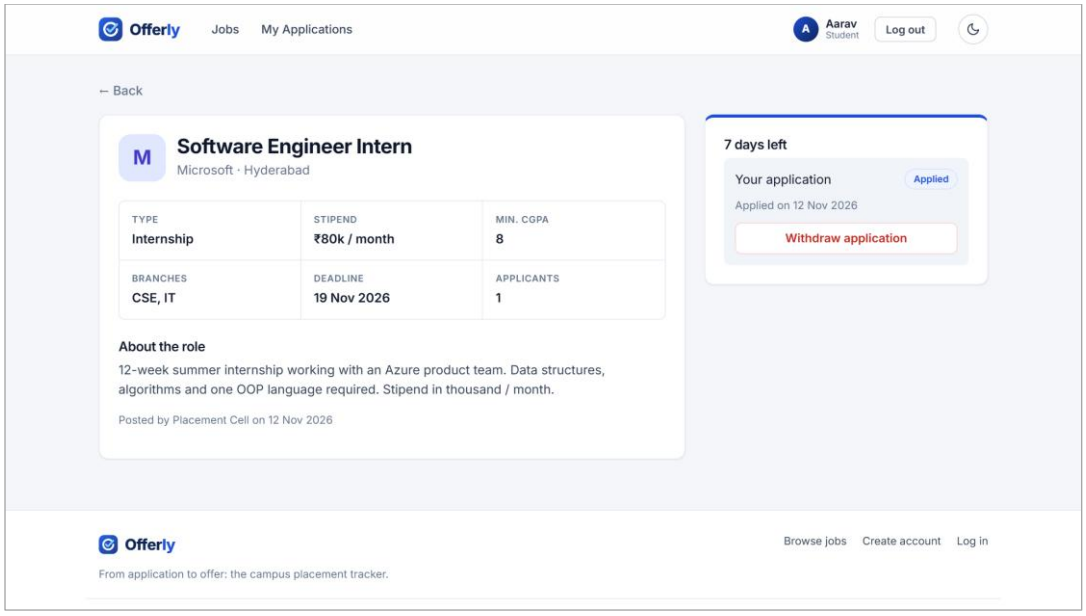


Figure A2.6. Job details after applying, showing the application status

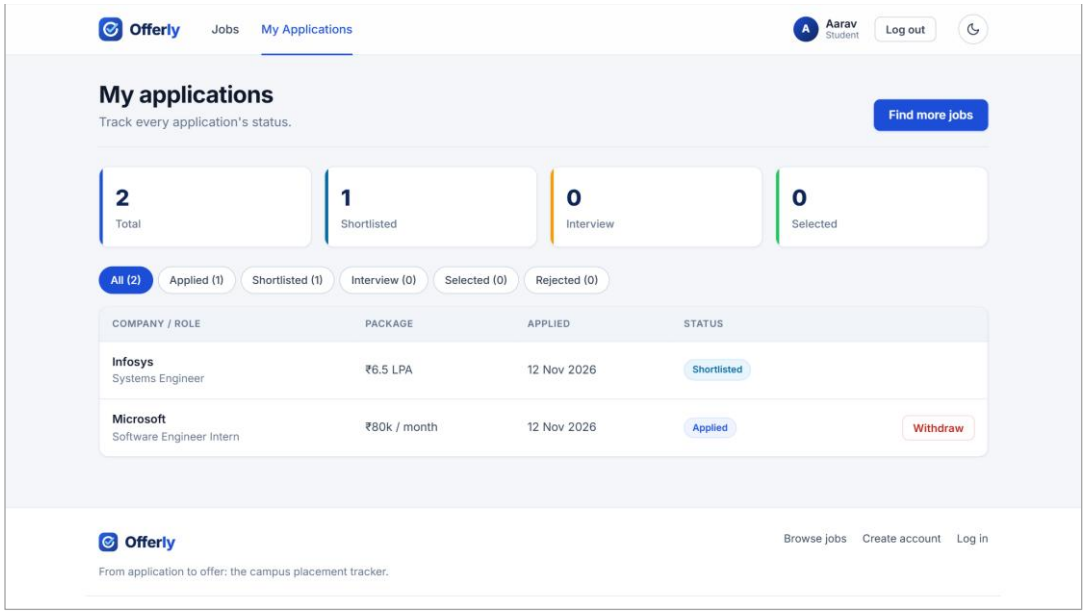


Figure A2.7. My Applications page with status tracking

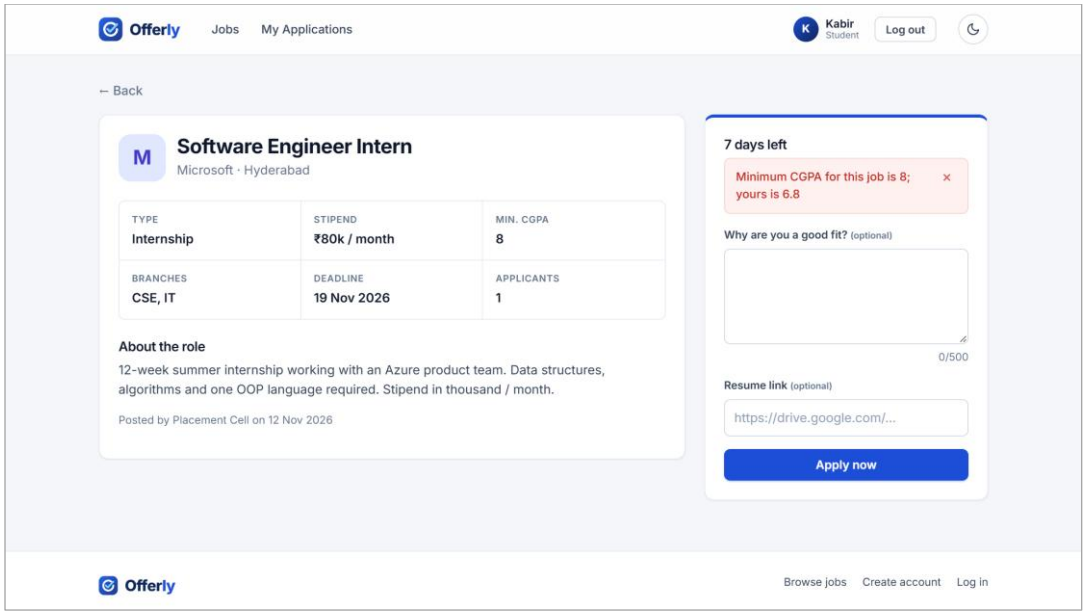


Figure A2.8. Application rejected by the CGPA eligibility rule

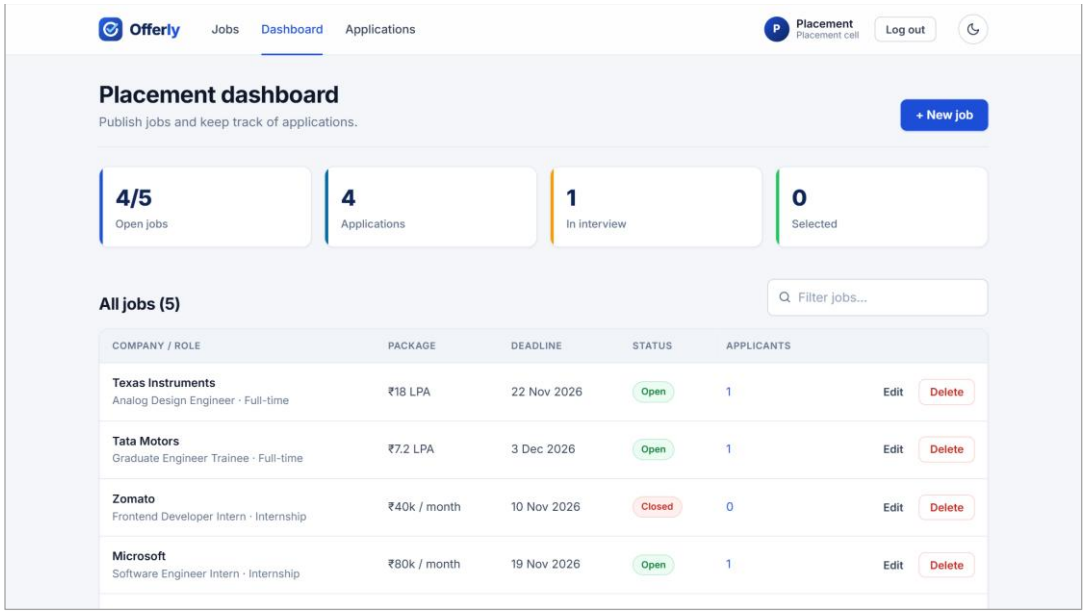


Figure A2.9. Placement-cell dashboard with statistics and job management

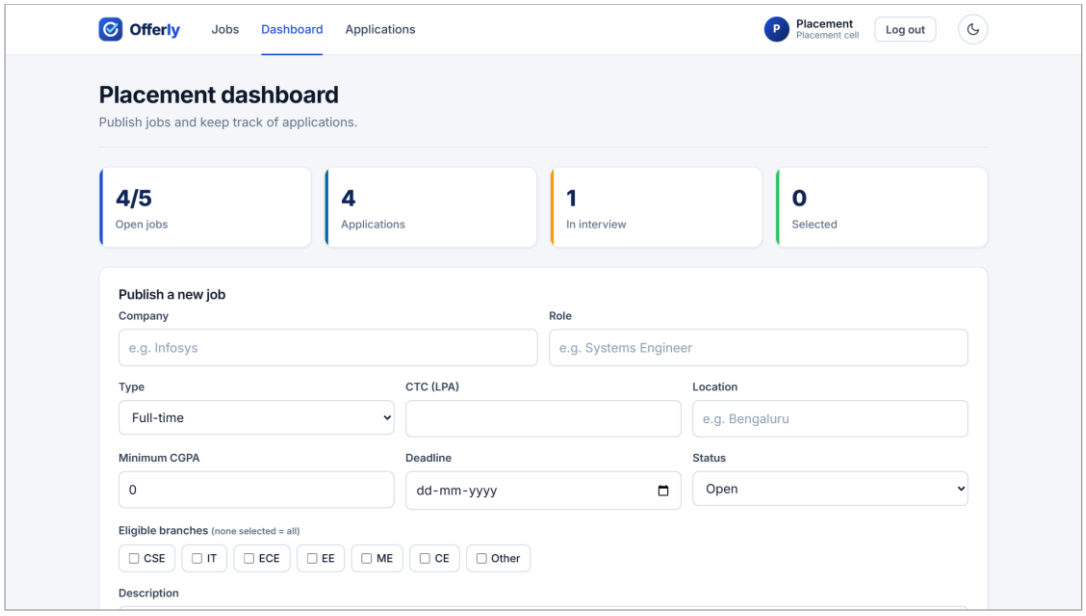


Figure A2.10. Form to publish a new job

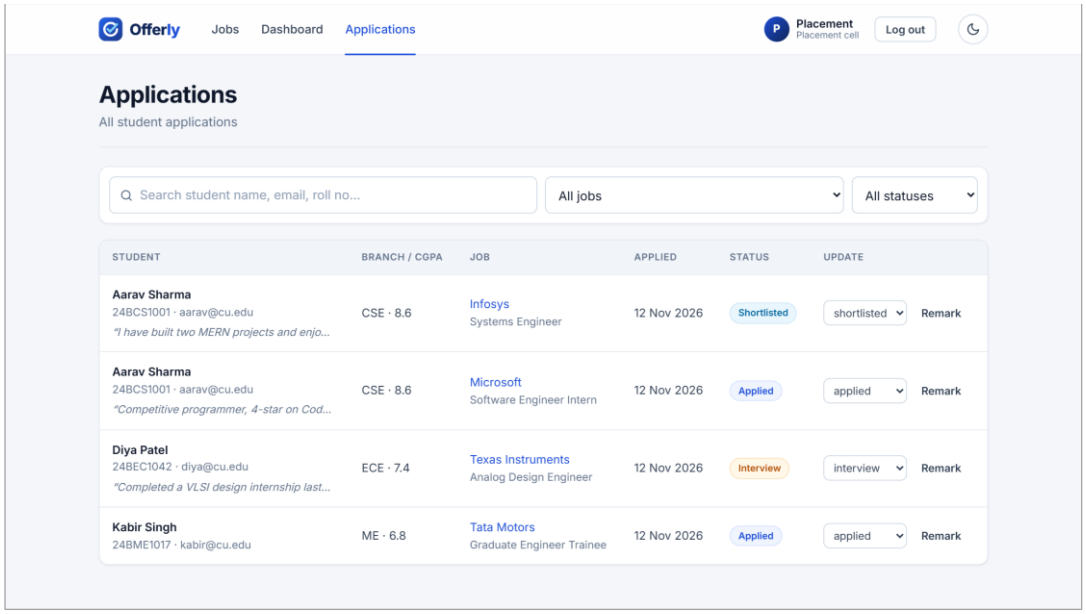


Figure A2.11. All applications with status updates

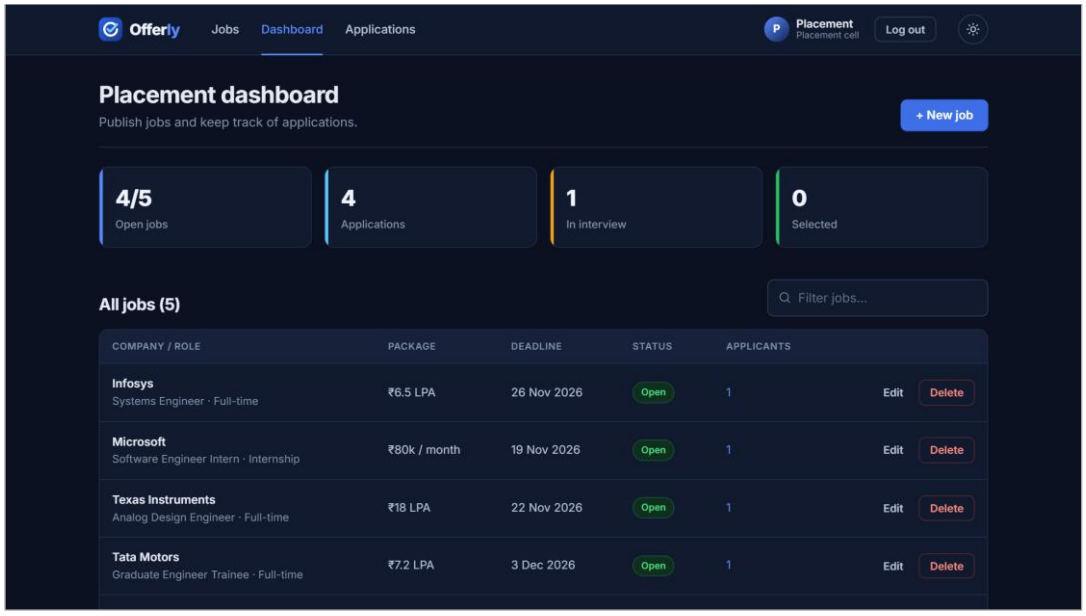


Figure A2.12. Dashboard in dark mode

APPENDIX 3 – SOURCE STRUCTURE AND ACHIEVEMENTS

A3.1 Source code structure

The repository (https://github.com/Yash1911ind/offerly) contains separate server and client folders, a README with setup instructions, and this report.

```
offerly/
├── server/
│   └── src/
│       ├── server.js          entry point: env, DB connection, listen
│       ├── app.js             Express app, routes, static client
│       ├── config/db.js       MongoDB connection
│       ├── models/            User.js, Job.js, Application.js
│       ├── middleware/        auth.js (protect, authorize), error.js
│       ├── routes/            auth.js, jobs.js, applications.js
│       ├── seedData.js        demo users, jobs, applications
│       ├── seed.js            npm run seed
│       └── devMemory.js       npm run server:memory
├── client/
│   └── src/
│       ├── api/client.js      fetch() wrapper with JWT
│       ├── context/           AuthContext.jsx
│       ├── components/        Navbar, JobCard, JobForm, ProtectedRoute, ...
│       ├── pages/             Home, Jobs, JobDetail, MyApplications, Admin...
│       └── index.css          light and dark themes, responsive layout
├── docs/                      project report
└── README.md
```

A3.2 Achievements

- A working MERN application covering all nine laboratory experiments in one real-world scenario.
- Three related MongoDB collections with schema validation, references and .populate().
- Secure authentication with bcrypt and JWT, plus role-based and ownership-based authorisation.
- The one-application-per-job rule enforced at both API and database level, verified under concurrent requests.
- Automatic deadline and CGPA/branch eligibility checks that remove manual screening work.
- 29 of 29 automated API tests passing.
- A responsive, branded interface (Offerly) with light and dark themes.